What is Ansible?

What is it Good At?

- **Configuration Management**: Generally used to manage on-prem / hybrid cloud server configurations.
- **Provisioning**: If you need spin up ***n*** copies of something in the same way or templated way – ensure it's done once and correctly.
- **Change/State Management**: Ansible checks to see if the state of the machine is the same as it's defined; if it's not, it updates it.
- **Simplicity**: Everything is managed in config files, everything runs in order, code is the documentation.

Fundamental Design

- **Goals:** They are exactly how they sound – something you want to do at a high-level i.e. deploy a web application.
- **Task Groups** : A collection of ***Tasks*** which execute to reach a ***Goal*** i.e. configure runtime environment, deploy application, expose deployment.
- **Tasks**: A ***Task*** is an operation where you define a step in a process. This can be thought as a step in a README, or in a method.
- **Handlers**: The difference between Handlers and Tasks, is that a ***Handler*** is triggered when a ***Task*** results in a state change – a ***Handler*** is then run as a result. To note handlers run at the **end of a play** and **only run once** even if triggered multiple times.

How Could We Classify an Application Deployment?

Task Group 1: Prepare the Server

- Task: Update the package manager cache
- Task: Install required dependencies (e.g., Python, Nginx)
- Task: Create application directories

Task Group 2: Deploy the Application

- Task: Pull the latest code from the repository
- Task: Install application dependencies (e.g., pip install)
- Task: Write/template the application config file

Task Group 3: Start & Expose the Service

- Task: Enable the application service
- Task: Configure Nginx as a reverse proxy

Handlers (triggered by state changes above):

- If the config file *changed* → Restart the application service
- If Nginx config *changed* → Reload Nginx

Playbooks

The Core Idea

In Ansible, a **playbook** is the file where you define *what you want to happen*. A playbook is written in YAML and reads almost like a human-readable runbook. This ties back to your earlier point — *code is the documentation*. You should be able to hand a playbook to someone unfamiliar with the system and they'd understand the intent.

The structure of a playbook is made up of one or more **Plays**. Each Play:

- Targets a set of hosts (e.g., webservers, databases)
- Defines a sequence of Task Groups / Tasks to execute against them
- Declares any Handlers that should fire on state changes

Order of execution is important — Ansible executes a playbook in a very deliberate order:

- All **Tasks** run top to bottom, in sequence
- Any triggered **Handlers** run at the end, once

There is no parallelism within a play by default — which again supports the *simplicity* principle from your Ansible overview.

A playbook is **idempotent** — you can run it repeatedly and it will only make changes when the actual state of a machine drifts from the desired state.

How it connects?

A playbook is the *container* for everything you've defined so far:

- Your **Goal** is the overall intent of the playbook (e.g., *"Deploy a Web Application"*)
- Your **Task Groups** map directly to logical sections within a Play
- Your **Tasks** are the individual steps executed in sequence within those groups
- Your **Handlers** sit declared within the Play, waiting to be triggered by a Task state change

Inventories

The Core Idea

In Ansible, an **inventory** is simply a list of the machines (hosts) you want to manage. Rather than hardcoding a server address into your playbooks, you define your hosts in an inventory file and reference them by group name. This keeps your playbooks reusable across different environments.

Hosts can be organised into groups, for example:

- `[webservers]` — all your Nginx/Apache hosts
- `[databases]` — all your Postgres/MySQL hosts
- `[production]` vs `[staging]` — environment-based splits

A host can belong to multiple groups, and groups can contain other groups (nested groups).

- **Static inventory** — a simple file (INI or YAML format) where you manually list hosts. Good for stable, on-prem environments.
- **Dynamic inventory** — a script or plugin that queries an external source (e.g., AWS, Azure, GCP) and returns hosts at runtime. Essential when infrastructure is constantly changing.

How it connects?

An inventory answers the question *"where does the Goal run?"* Your Task Groups and Tasks define *what* to do, but the inventory defines *which machines* those Tasks execute against.